



COMUNICAÇÃO DISTRIBUÍDA UTILIZANDO O MIDDLEWARE ZEROMQ

*Bruno Stanley , Larissa Faustino
Patricki Hernandez, Patricky Costa*



Objetivo

Demonstrar como o middleware ZeroMQ simplifica a comunicação entre aplicações distribuídas por meio do paradigma **brokerless messaging**, destacando desempenho, baixo acoplamento e implementação prática.

Sumário

- Introdução e Conceitos Fundamentais
- Arquitetura e Funcionamento do ZeroMQ
- Implementação do código
- Demonstração e Resultados
- Conclusão



Introdução e Conceitos Fundamentais

Os **desafios** dos sockets de baixo nível, como alto acoplamento e complexidade na comunicação distribuída, motivam o uso de **middlewares** como solução para abstração, escalabilidade e manutenção dos sistemas.



A Tecnologia ZeroMQ

O ZeroMQ, desenvolvido por Pieter Hintjens, é uma biblioteca de **mensagens assíncronas** criada para facilitar a comunicação entre aplicações distribuídas. Diferente dos sockets tradicionais, ele abstrai detalhes complexos da comunicação em rede, tornando a troca de mensagens mais **simples, rápida e escalável**. *(Pamadi et al., 2020)*

Sua arquitetura dispensa a necessidade de um **broker central**, permitindo que **os próprios sistemas se comuniquem diretamente**. Isso reduz latência, consumo de recursos e pontos únicos de falha, além de aumentar a flexibilidade da aplicação.





Paradigma Brokerless Messaging

Cada nó possui maior **autonomia** na comunicação, estabelecendo conexões diretas com outros processos da aplicação.

Este paradigma favorece o **desacoplamento** entre componentes distribuídos. Os processos podem se comunicar de forma assíncrona, sem depender da sincronização **temporal** e sem a necessidade de conhecer diretamente a localização ou implementação dos demais componentes, permitindo um desacoplamento **referencial**. *(Pamadi et al., 2020)*

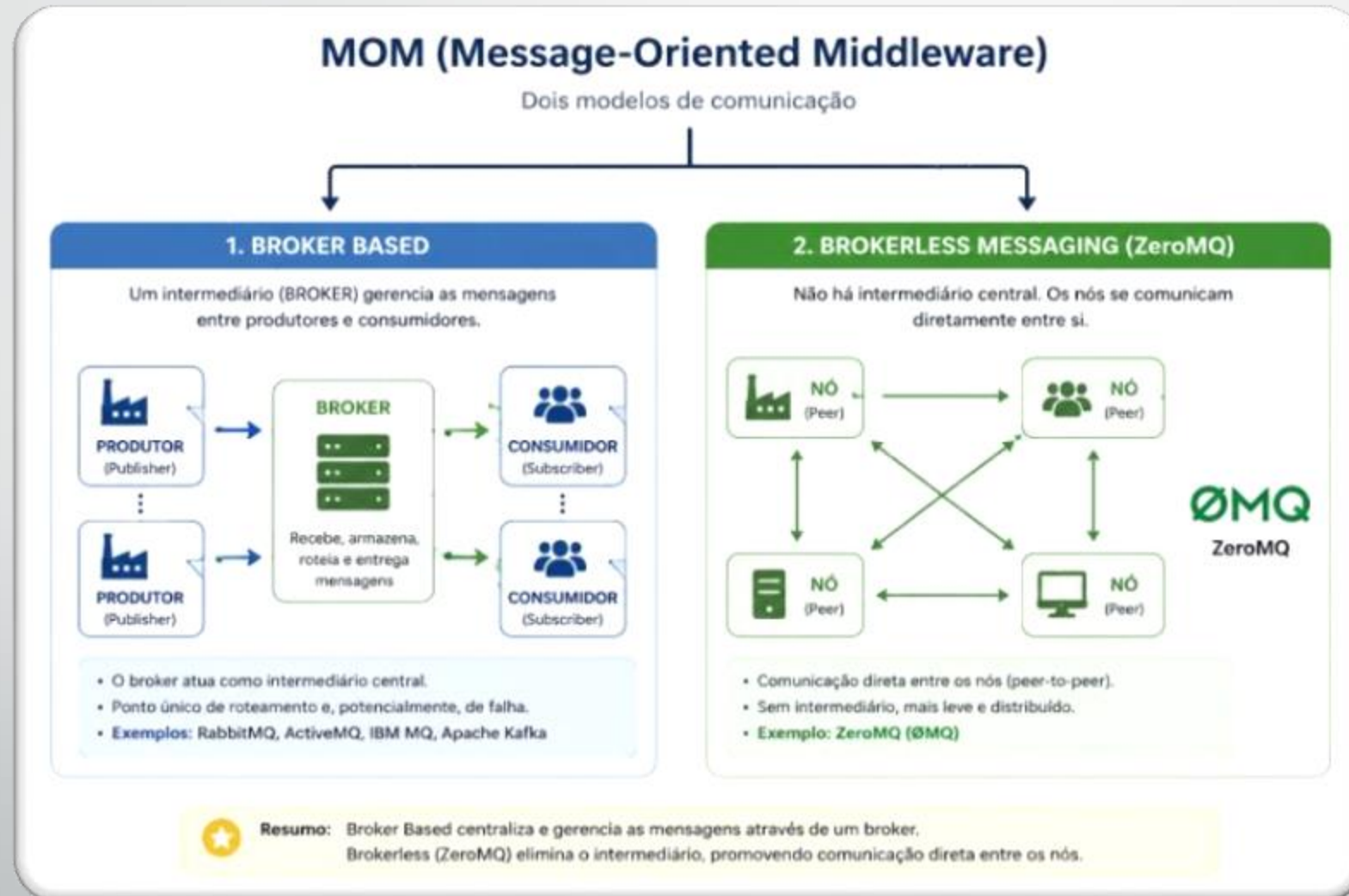


Principais características do Paradigma Brokerless Messaging

- **Baixa latência**: comunicação direta entre processos reduz o tempo de resposta
- **Alto throughput**: capacidade de processar grandes volumes de mensagens simultaneamente.
- **Zero-copy**: otimização no uso de CPU e memória durante a transmissão de dados
- **Escalabilidade horizontal**: ausência de ponto central de gargalo, permitindo crescimento do sistema sem degradação proporcional de desempenho.

(Pamadi et al., 2020)

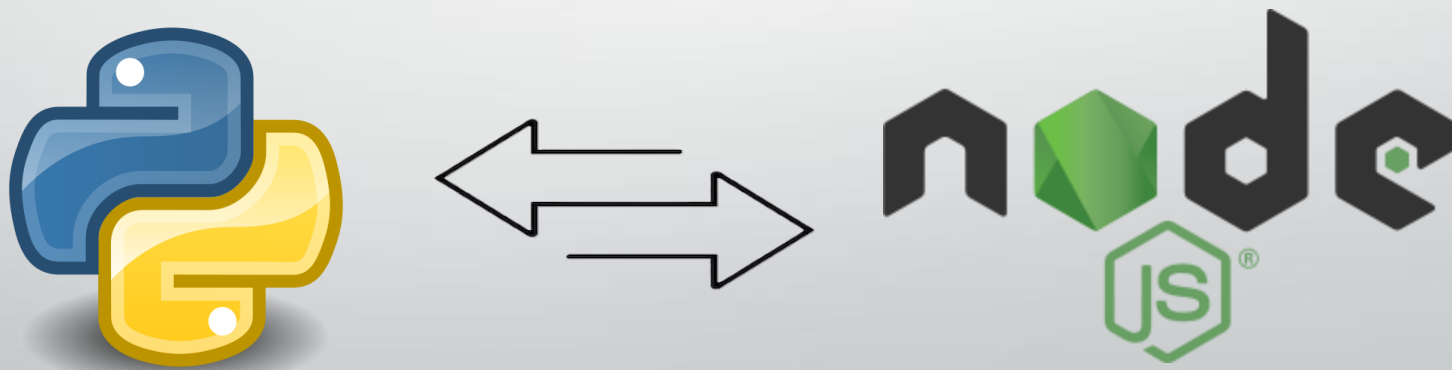
Representação do Modelo de Comunicação MOM





Arquitetura Cliente-Servidor Heterogênea: Interoperabilidade do Sistema

A **interoperabilidade** do sistema foi alcançada por meio da comunicação entre aplicações desenvolvidas em linguagens diferentes — **Python** no cliente e **Node.js** no servidor. Essa abordagem permitiu troca de mensagens padronizadas e integração transparente entre **plataformas distintas**.



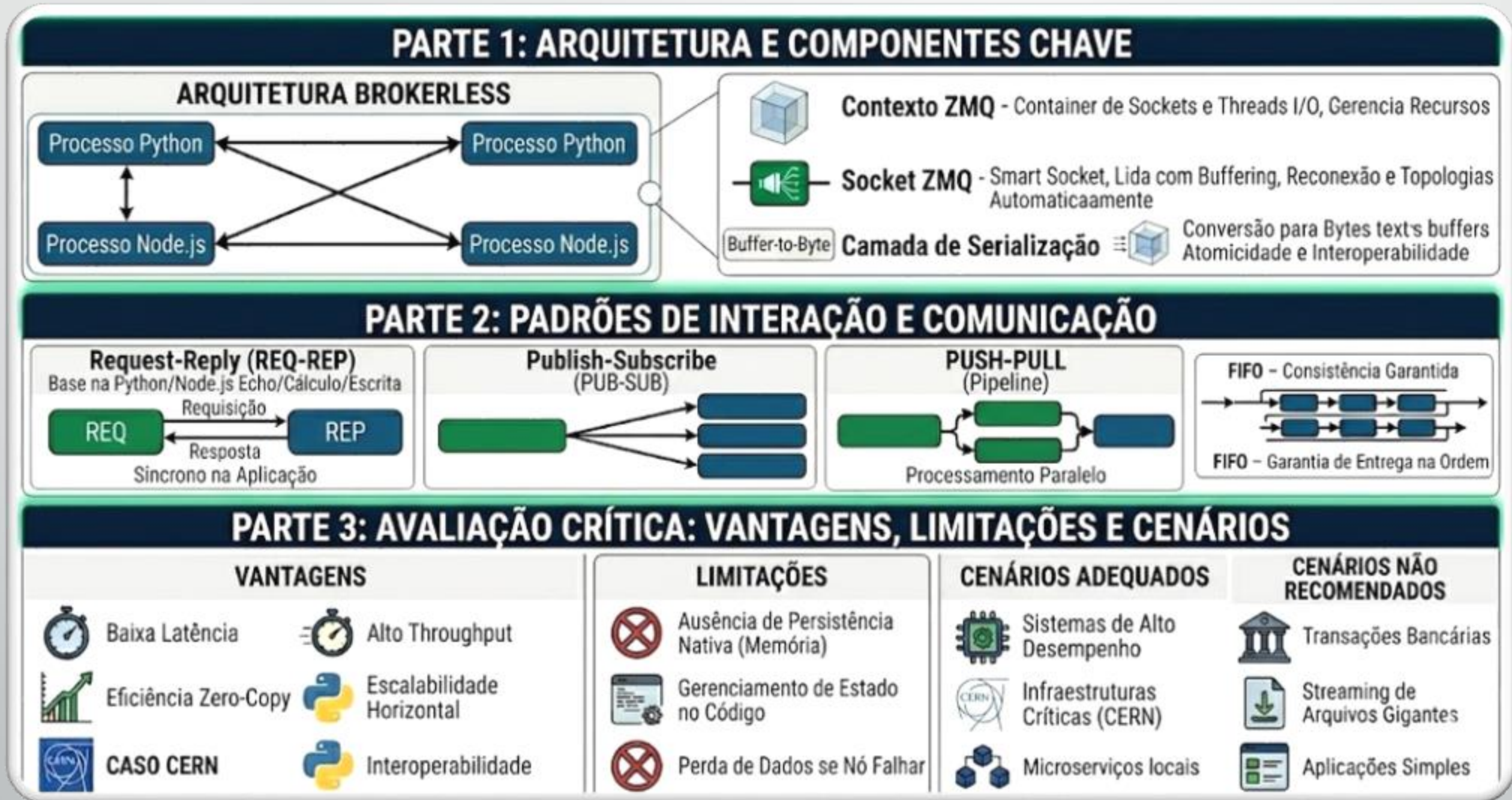


Arquitetura e Funcionamento do ZeroMQ

A tecnologia abstrai sockets tradicionais por meio de componentes como **Context e sockets ZeroMQ**, responsáveis pelo gerenciamento de conexões, filas e troca de mensagens. Como também, suporta comunicação assíncrona, múltiplos padrões de interação distribuída — como **Request/Reply, Publish/Subscribe e Push/Pull**. (Lauener & Sliwinski, 2017)

O modelo de consistência do sistema é baseado no ordenamento **FIFO** — garantindo que as mensagens sejam entregues ao servidor na mesma ordem em que foram enviadas pelo cliente.

Representação da Arquitetura e Componentes Chave



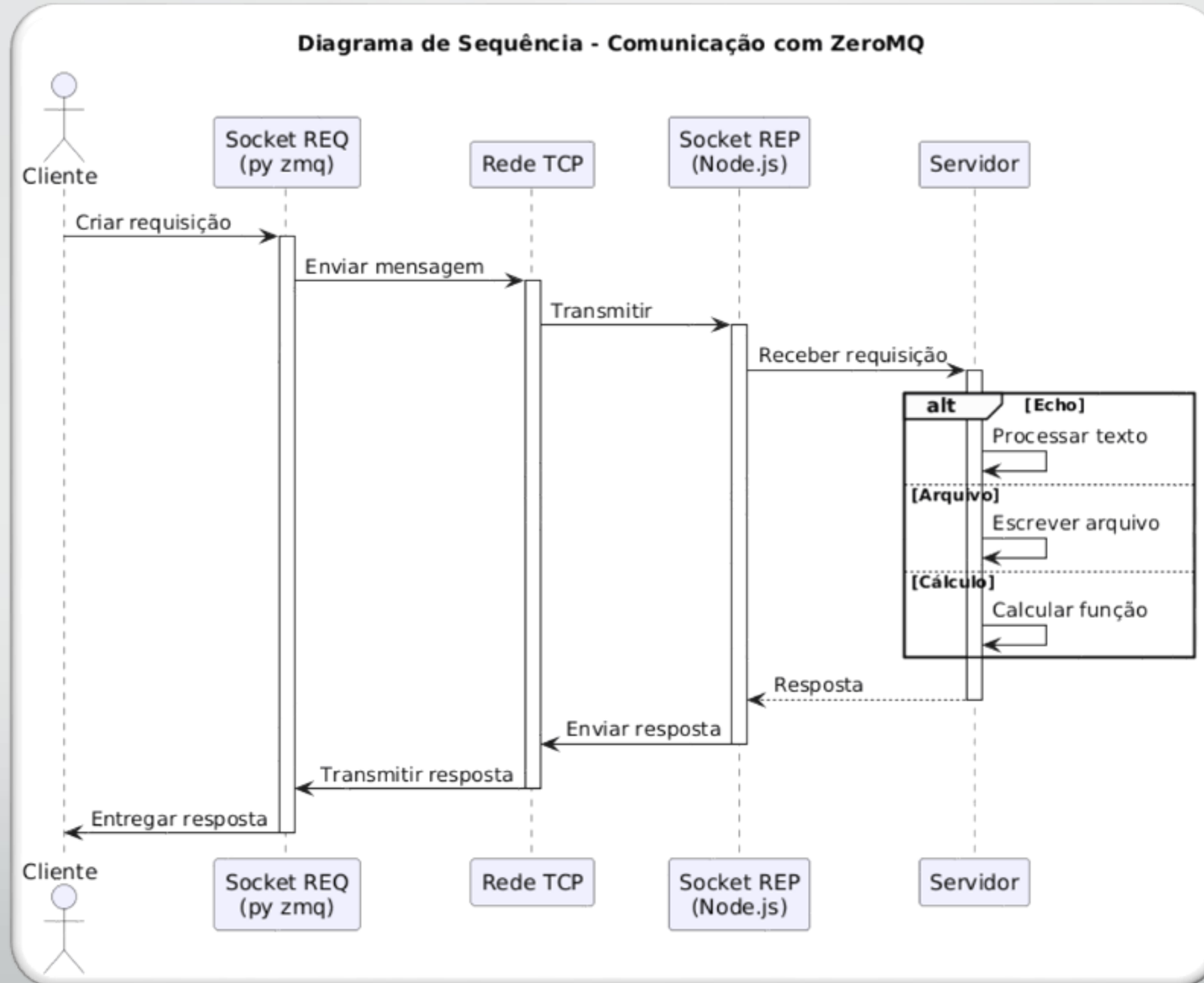


Modelagem da Solução e Fluxo de Comunicação

A solução foi modelada separando a lógica de aplicação da comunicação em rede. O cliente envia requisições em **formato textual**, o ZeroMQ realiza o transporte via **TCP**, o servidor processa a operação solicitada e retorna a resposta utilizando o padrão **Request/Reply**.

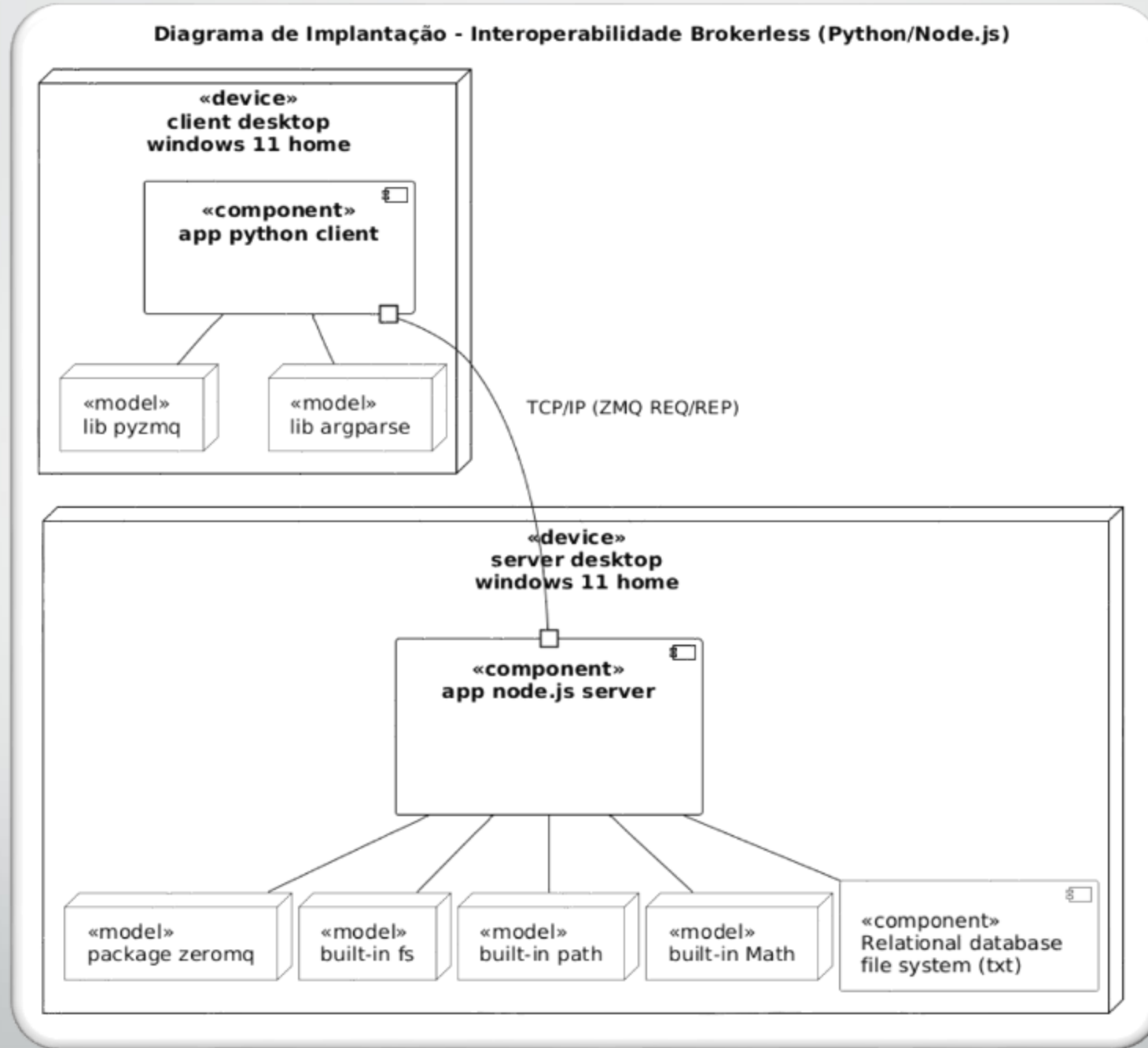


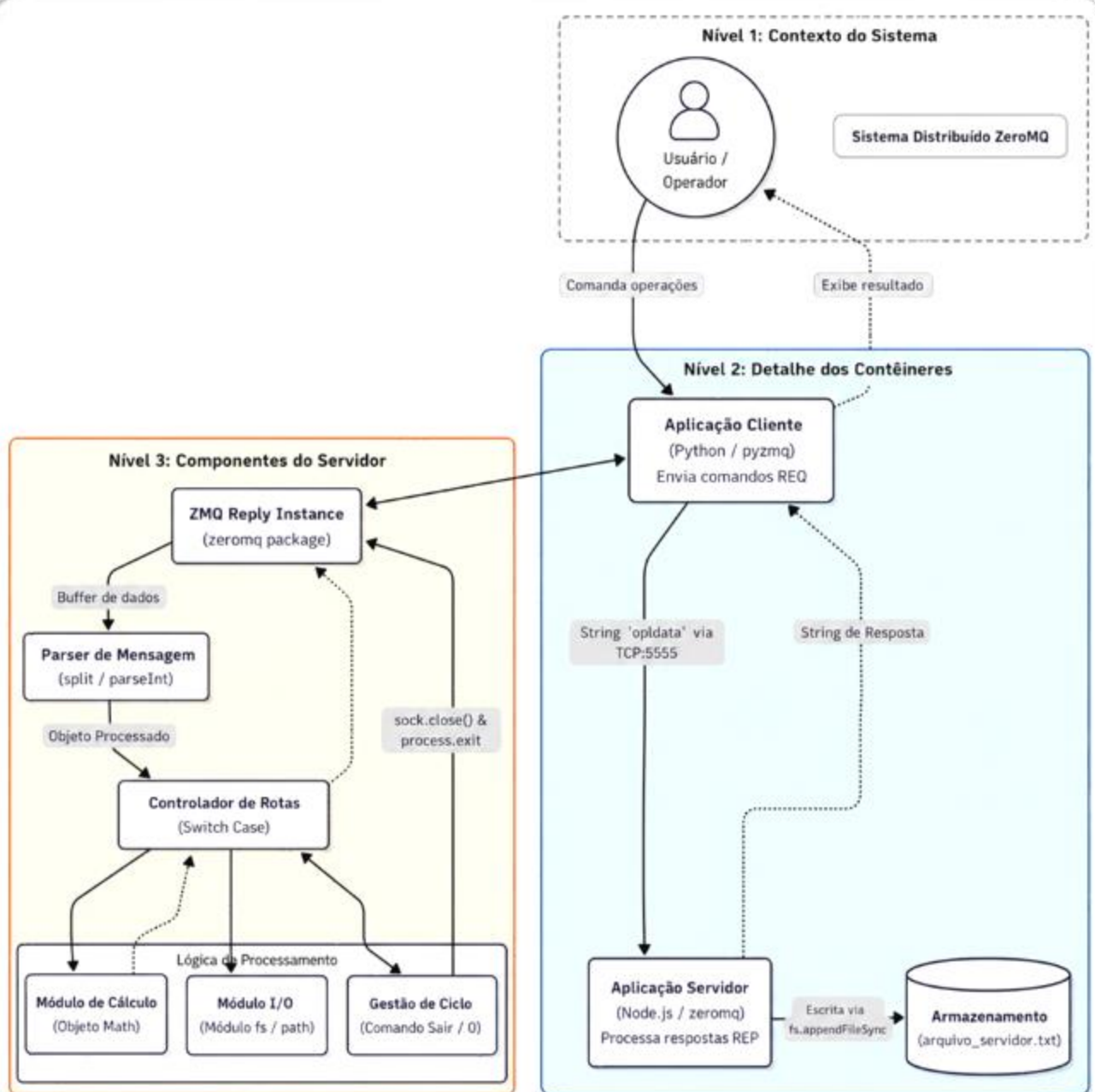
Fluxo de
comunicação entre
cliente e servidor



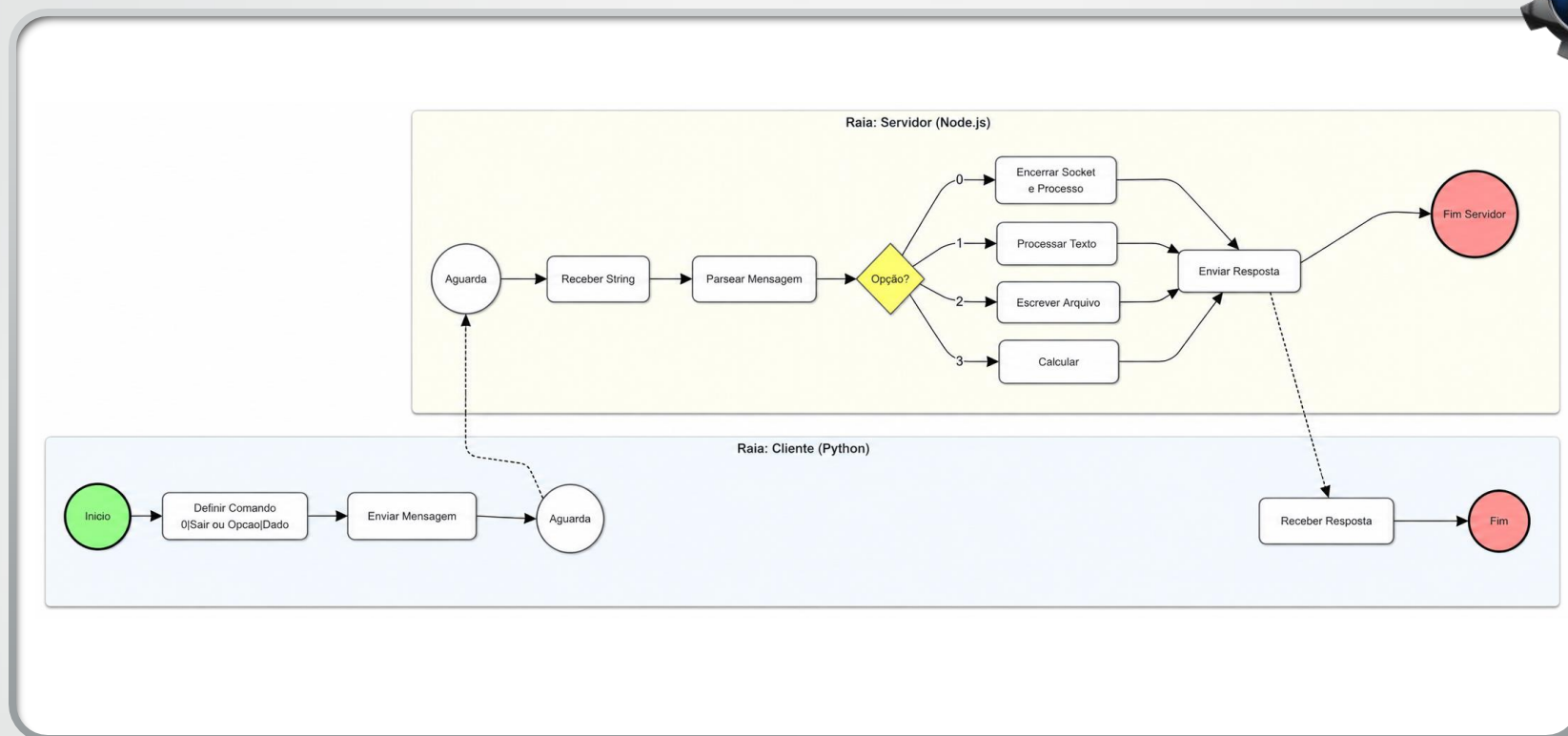


Distribuição dos
componentes
representando a
Interoperabilidade





Representação da arquitetura do sistema em diferentes níveis de abstração através do **modelo C4**



Fluxo completo do sistema representado pelo
Diagrama BPMN



Implementação do Código

Divisão entre cliente em **Python** e servidor em **Node.js**, cada um utilizando **bibliotecas específicas** para suas responsabilidades:

Cliente (Python)

- **pyzmq** → Comunicação com o servidor utilizando socket REQ
- **argparse** → Leitura de parâmetros via linha de comando

Servidor (Node.js)

- **zeromq** → Implementação do socket REP e gerenciamento da comunicação
- **fs** → Persistência de dados em arquivo .txt
- **Math** → Execução das operações matemáticas requisitadas pelo cliente



Funcionamento do Servidor Node.js

O servidor Node.js inicializa o socket REP, define o endpoint TCP e executa o **bind**, permanecendo em escuta para **receber e processar** requisições do cliente.

```
117  const argumentos = lerArgumentos();  
118  iniciarServidor(argumentos.host, argumentos.port);
```

```
27  async function iniciarServidor(host, port) {  
28      const sock = new zmq.Reply();  
29      const endpoint = `tcp://${host}:${port}`;  
30      await sock.bind(endpoint);  
31  
32      console.log(`Servidor Node.js rodando em ${endpoint}...`);
```



Funcionamento do Cliente Python

O cliente inicializa o **contexto** ZeroMQ, cria o **socket REQ** e estabelece conexão com o servidor via TCP, permitindo o envio de requisições e o recebimento de respostas.

```
19 class ClienteZeroMQ:
20     def __init__(self, host:str, port:int):
21         self.host = host
22         self.port = port
23         self.context = zmq.Context().instance()
24         self.socket = self.context.socket(zmq.REQ)
25         self.endpoint = f"tcp://{self.host}:{self.port}"
```



Interpretação e Direcionamento das Requisições

As mensagens trocadas entre cliente e servidor seguem um protocolo simples baseado em texto, no formato **opcao|payload**, no qual a **opção** indica a operação desejada e o **payload** contém os dados necessários.

Opção	Operação	Payload	Exemplo enviado
1	Mensagem de texto	Texto livre	1 Hello
2	Gravar em arquivo	Texto para salvar	2 Olá mundo
3	Cálculo matemático	função,parâmetros	3 quadrado,5

```
server.js
34 for await (const [msg] of sock) {
35   const [opcaoStr, payload]
    = msg.split("|");
36   const opcao = parseInt(opcaoStr);
37   switch (opcao) {
38     case 1: // texto
39     case 2: // arquivo
40     case 3: // cálculo
41   }
42   await sock.send(resposta);
43 }
```



Processamento das Requisições

O servidor recebe a mensagem do cliente, extrai a opção e o payload e direciona a requisição para a operação correspondente, retornando a resposta ao cliente.

```
server.js

switch (opcao) {
  case OPCA0_TEXTO:
    if (payload === "Hello") resposta = "World";
    else resposta = "Mensagem desconhecida";
    break;
  case OPCA0_ARQUIVO:
    fs.appendFileSync("arquivo_servidor.txt", payload + "\n");
    resposta = "Arquivo atualizado com sucesso.";
    break;
  case OPCA0_CALCULO:
    resposta = calcular(payload);
    break;
}
```



Demonstração e Resultados



Conclusão

ZeroMQ demonstrou eficiência na comunicação entre *aplicações heterogêneas*, oferecendo baixo acoplamento, simplicidade e baixa latência por meio do modelo *brokerless*. Entretanto, limitações como a ausência de persistência nativa e garantias fortes de entrega restringem seu uso em aplicações críticas, motivando comparações futuras com **RabbitMQ e Apache Kafka**.



Referências

- **LAUENER, J.; SLIWINSKI, W.** *How to Design & Implement a Modern Communication Middleware Based on ZeroMQ*, 2017.
- **PAMADI, V. N.** et al. *Comparative Analysis of gRPC vs. ZeroMQ*, 2020.
- **TANENBAUM, A. S.; VAN STEEN, M.** *Distributed Systems: Principles and Paradigms*, 2007.
- **ZEROMQ COMMUNITY.** *Getting Started with ZeroMQ*, 2026.